1 Como usar este script

2 Preparo

3 Dados

4 Pré-processamento

4.1 Outliers nas bandas

4.2 Extração dos índices

5 Seleção dos índices: regressão ReliefF

6 Os 14 métodos

7 Treinamento (80%) e avaliação no conjunto de validação (20%)

8 Aplicação nos conjuntos externos

9 Por que alguns métodos se adaptam melhor?

10 Resumo para o artigo

11 Limites do exemplo

Predição do rendimento de grãos de soja a partir de fotos de folíolos (Artigo 4)

Exemplo reproduzível do roteiro de análise: índices RGB, ReliefF, 14 métodos, Gradient Boosting e validação em dois conjuntos externos

Cristhian Milbradt Babeski

08/10/2026

1 Como usar este script

Um único arquivo. `source("exemplo_artigo4_foliolos.R")` roda e imprime só resultados no console; `rmarkdown::render("exemplo_artigo4_foliolos.R")` gera este relatório (o texto em `#'` vira Markdown, o código aparece abaixo de cada explicação). Cada tabela e figura tem legenda numerada. Para usar os seus dados, salve um Excel com as colunas descritas na aba `dicionario` (aba `dados`) e troque `CFG$arquivo`. Com acentos estranhos: `source(..., encoding = "UTF-8")`.

O que é o exemplo. Segue o roteiro do Artigo 4: prever o **rendimento de grãos por hectare (RGHA, kg ha^{-1})** usando só a cor de fotos de folíolos. Os dados são **simulados** com a estrutura do artigo; os números servem para mostrar o método, não como resultado.

Conjunto	Papel	Unidade	Fotos (n)
I	construção: treino (80%) e validação (20%); cultivar URNRS 23	parcela	≈ 13,4 mil (Artigo: 13.447)
II	testemunhas, outro ambiente, mesma safra	15 cultivares × (2 repetições × 3 folíolos)	90
III	linhagens F5 da cultivar, mesmo ambiente do conjunto I	215 linhagens × (2 × 3)	1.290

Fluxo: (1) separar R, G e B de cada foto, (2) remover outliers, (3) calcular os índices com `Breed_RGB`, (4) escolher os índices por **regressão ReliefF** (Artigo: VARI e ExGR), (5) dividir 80/20, (6) treinar **14 métodos**, (7) avaliar com validação de 20 partes estratificadas e 10 repetições 80/20 (R^2, RMSE, MAE, MAPE), (8) escolher o método (Artigo: **Gradient Boosting**), (9) aplicar nos conjuntos II e III, (10) diagnosticar **por que alguns métodos se adaptam melhor**.

2 Preparo

Pacotes, configuração e funções auxiliares. `quiet()` esconde avisos dos pacotes para o console mostrar só resultados. Instale antes o que faltar: `install.packages(c("openxlsx","ggplot2","patchwork","caret","ranger","glmnet","mgcv","FNN","e1071","gbm","arm","nnet"))`. `xgboost` é opcional: se estiver instalado, entra como 14º método.

```
precisa <- c("openxlsx", "ggplot2", "patchwork", "caret", "ranger", "glmnet", "mgcv", "FNN", "e1071", "gbm",
"arm", "nnet")
faltam <- precisa[!sapply(precisa, requireNamespace, quietly = TRUE)]
if (length(faltam)) stop("Instale: ", paste(faltam, collapse = ", "))
suppressPackageStartupMessages(library(ggplot2))
tem_xgb <- requireNamespace("xgboost", quietly = TRUE)

CFG <- list(
  semente = 123, arquivo = "dados_artigo4.xlsx", saidas = "saidas_artigo4",
  rapido = FALSE,                       # TRUE: 5 partes e 3 repetições (teste rápido); FALSE: 20 partes e 1
0 repetições, como no Artigo
  n_folds = 20, n_rep = 10, prop_teste = 0.20,
  ## simulação (só usada se o Excel não existir)
  n_parcelas_I = 464, fotos_por_parcela = 30, n_cultivares_II = 15, n_linhagens_III = 215, reps = 2, foliolo
s = 3,
  ## pré-processamento
  iqr_k = 1.5, exgr = "diferenca",      # "diferenca": ExGR = ExG - ExR (padrão da literatura); "razao": fór
mula original do Breed_RGB
  n_relief = 4000, k_relief = 30,       # ReliefF: instâncias sorteadas e vizinhos
  metodo_final = "Gradient Boosting",  # método do Artigo 4; o script mostra qual teve o menor RMSE e quais
empatam com ele
  indices_modelo = c("VARI", "ExGR"),   # índices do Artigo 4. NULL = usa os 2 primeiros do ReliefF (sem par
es com |r| > colin_r_max)
  n_indices = 2, colin_r_max = 0.95,
  ## Gradient Boosting do Artigo (scikit-learn -> gbm)
  gb = list(arvores = 100, taxa = 0.10, profundidade = 1, subamostra = 1.0, min_obs_no = 3))
if (CFG$rapido) { CFG$n_folds <- 5; CFG$n_rep <- 3 }
dir.create(CFG$saidas, showWarnings = FALSE)
knitando <- isTRUE(getOption("knitr.in.progress"))

options(width = 170)
quiet <- function(expr) { suppressWarnings(suppressMessages({ capture.output(r <- expr); r })) }
n_tab <- 0; n_fig <- 0
mostra <- function(x, legenda = NULL, d = 2) {
  x[] <- lapply(x, function(v) if (is.numeric(v)) round(v, d) else v)
  if (knitando) { n_tab <<- n_tab + 1; return(knitr::kable(x, row.names = FALSE, caption = paste0("**Tabela
", n_tab, ".** ", legenda))) }
  if (!is.null(legenda)) cat("\n", strsplit(legenda, "\\. ")[[1]][1], "\n", sep = "")
  print(x, row.names = FALSE); invisible(x)
}
fig_md <- function(nome, legenda) {
  if (!knitando) return(invisible(NULL))
  n_fig <<- n_fig + 1
  knitr::asis_output(sprintf("\n\n![**Figura %d.** %s](%s){width=100%%}\n\n", n_fig, legenda, file.path(CFG
$saidas, nome)))
}
mostra_fig <- function(g, nome, legenda, w = 9, h = 4.5) { ggsave(file.path(CFG$saidas, nome), g, width = w,
height = h, dpi = 200, bg = "white"); fig_md(nome, legenda) }
salva <- function(x, nome) write.csv(x, file.path(CFG$saidas, paste0(nome, ".csv")), row.names = FALSE)
texto <- function(...) { s <- paste0(...); if (knitando) knitr::asis_output(paste0("\n\n", s, "\n\n")) else
{ cat("\n", gsub("\\*\\*|`", "", s), "\n", sep = ""); invisible(NULL) } }
```

2.1 Métricas e termos técnicos

Seja y_i o RGHA observado, $\hat{y}_i$ o predito e n o número de fotos avaliadas.

- **R²** $= 1 - \sum(y_i - \hat{y}_i)^2 / \sum(y_i - \bar{y})^2$: compara o modelo com prever sempre a média. Depende da variância de y no conjunto avaliado: com faixa estreita de rendimento ele é baixo mesmo com erro pequeno.
- **RMSE** $= \sqrt{n^{-1}\sum(y_i - \hat{y}_i)^2}$ e **MAE** $= n^{-1}\sum|y_i - \hat{y}_i|$, em kg ha⁻¹. O RMSE pune mais os erros grandes.

- **MAPE** $= n^{-1} \sum |y_i - \hat{y}_i| / y_i$: erro relativo médio; o Artigo traz como fração (0,238), aqui em %.
- **ReliefF para regressão (RReliefF):** para cada foto sorteada, olha os k vizinhos no espaço dos índices e pontua o índice que **muda junto com o RGHA**: sobe se vizinhos com índice diferente têm RGHA diferente, desce se têm RGHA parecido. Capta relações não lineares e interações.
- **VIF** do índice j: $1/(1 - R_j^2)$, com R_j^2 da regressão desse índice nos demais; > 10 indica redundância forte.
- **IQR 1,5:** valor é outlier se está fora de $[Q_1 - 1{,}5\,IQR,\ Q_3 + 1{,}5\,IQR]$.
- **Estratificar:** dividir (ou fazer as partes da validação cruzada) mantendo em cada parte a mesma distribuição do RGHA (por quantis).
- **Gradient Boosting:** soma árvores pequenas em sequência, cada uma corrigindo o erro das anteriores: $F_m = F_{m-1} + \nu\, h_m$, com ν a taxa de aprendizado. Com **profundidade 1** cada árvore é um único corte (*stump*): o modelo vira uma soma de curvas em degraus, $F(x) = \sum_j f_j(x_j)$.

```
rmse <- function(o, p) sqrt(mean((o - p)^2))
mae  <- function(o, p) mean(abs(o - p))
mape <- function(o, p) 100 * mean(abs(o - p) / o)
r2   <- function(o, p) 1 - sum((o - p)^2) / sum((o - mean(o))^2)
metricas <- function(o, p) data.frame(R2 = r2(o, p), RMSE = rmse(o, p), MAE = mae(o, p), MAPE = mape(o, p))
```

2.2 Índices RGB

O que é o `Breed_RGB` ? Função do autor (fórmulas mantidas sem alteração): recebe uma tabela com as bandas médias de cada foto (colunas R, G, B) e devolve a tabela com 49 índices de cor acrescentados (VARI, GLI, NGRDI, TGI, ExG, ExR, MGRVI, HUE, entre outros). O Artigo cita 48; confira qual não entrou. Cada índice combina as três bandas para realçar verdor, lesão ou senescência.

Atenção ao `ExGR` . No `Breed_RGB` ele é uma razão, $(2G - R - B)/(1{,}4R - G)$, que explode quando $1{,}4R \approx G$. Na literatura o ExGR é a *diferença* $\mathrm{ExG} - \mathrm{ExR}$. O script usa a diferença por padrão (`CFG$exgr = "diferenca"`); confira qual fórmula o Artigo 4 usou.

```
Breed_RGB <- function(dados, R, G, B) {
  R <- substitute(R)
  G <- substitute(G)
  B <- substitute(B)
  dados$BG <- dados[[B]] - dados[[G]]
  dados$BGI <- dados[[B]] / dados[[G]]
  dados$BI <- sqrt((dados[[R]]^2 + dados[[G]]^2 + dados[[B]]^2) / 3)
  dados$BIM <- sqrt((dados[[R]]*2 + dados[[G]]*2 + dados[[B]]*2) / 3)
  dados$BR <- dados[[B]] - dados[[R]]
  dados$bx <- 0.55 * ((dados[[B]] - (0.2126 * dados[[R]] + 0.7152 * dados[[G]] + 0.0722 * dados[[B]])) / (1.
0 - 0.0722))
  dados$bxa <- (0.55 * ((dados[[B]] - (0.2126 * dados[[R]] + 0.7152 * dados[[G]] + 0.0722 * dados[[B]])) /
(1.0 - 0.0722))) -
    (0.55 * ((dados[[R]] - (0.2126 * dados[[R]] + 0.7152 * dados[[G]] + 0.0722 * dados[[B]])) / (1.0 - 0.212
6)))
  dados$CI <- (dados[[R]] - dados[[B]]) / dados[[R]]
  dados$CIVE <- (0.44 * dados[[R]] - 0.81 * dados[[G]] + 0.39 * dados[[B]]) / (dados[[R]] + dados[[G]] + dad
os[[B]]) + 18.79
  dados$ExB <- (1.4 * dados[[B]] - dados[[G]]) / (dados[[R]] + dados[[G]] + dados[[B]])
  dados$EXG <- 2 * (dados[[G]] * (dados[[R]] + dados[[G]] + dados[[B]]) - (dados[[R]] / (dados[[R]] + dados
[[G]] + dados[[B]])) - dados[[B]] / (dados[[R]] + dados[[G]] + dados[[B]]))
  dados$EXGRaw <- 2 * dados[[G]] - dados[[R]] - dados[[B]]
  dados$ExGR <- (2 * dados[[G]] - dados[[R]] - dados[[B]]) / (1.4 * dados[[R]] - dados[[G]])
  dados$ExR <- 1.4 * dados[[R]] - dados[[G]]
  dados$EXR <- (1.4 * dados[[R]] - dados[[G]]) / (dados[[R]] + dados[[G]] + dados[[B]])
  dados$ExRM <- (2 * dados[[R]] - dados[[G]] - dados[[B]]) / (dados[[R]] + dados[[G]] + dados[[B]])
  dados$GB <- dados[[G]] - dados[[B]]
  dados$GLAI <- (25 * (dados[[G]] - dados[[R]]) / (dados[[G]] + dados[[R]] - dados[[B]])) + 1.25
  dados$GLI <- ((dados[[G]] - dados[[R]]) + (dados[[G]] - dados[[B]])) / (dados[[G]] + dados[[R]] + dados
[[G]] + dados[[B]])
  dados$GLI2 <- (2 * dados[[G]] - dados[[R]] + dados[[B]]) / (2 * dados[[G]] + dados[[R]] + dados[[B]])
  dados$GR <- dados[[G]] - dados[[R]]
  dados$GRAY <- 0.299 * dados[[R]] + 0.587 * dados[[G]] + 0.114 * dados[[B]]
  dados$GRAY2 <- ((dados[[R]]^2.2 + (1.5 * dados[[G]])^2.2 + (0.6 * dados[[B]])^2.2) / (1 + 1.5^2.2 + 0.6^2.
2))^(1 / 2.2)
  dados$HI <- (2 * dados[[R]] - dados[[G]] - dados[[B]]) / (dados[[G]] - dados[[B]])
  dados$HUE <- atan(2 * (dados[[B]] - dados[[G]] - dados[[R]]) / (30.5 * (dados[[G]] - dados[[R]])))
  dados$I <- dados[[R]] + dados[[G]] + dados[[B]]
  dados$IGB <- dados[[G]] / dados[[B]]
  dados$IGR <- dados[[G]] / dados[[R]]
  dados$IRB <- dados[[R]] / dados[[B]]
  dados$L <- (dados[[R]] + dados[[G]] + dados[[B]]) / 3
  dados$Lx <- 0.2126 * dados[[R]] + 0.7152 * dados[[G]] + 0.0722 * dados[[B]]
  dados$Lxa <- (0.2126 * dados[[R]] + 0.7152 * dados[[G]] + 0.0722 * dados[[B]]) - (0.55 * ((dados[[R]] -
(0.2126 * dados[[R]] + 0.7152 * dados[[G]] + 0.0722 * dados[[B]])) / (1.0 - 0.2126)))
  dados$Lxb <- (0.2126 * dados[[R]] + 0.7152 * dados[[G]] + 0.0722 * dados[[B]]) - (0.55 * ((dados[[B]] -
(0.2126 * dados[[R]] + 0.7152 * dados[[G]] + 0.0722 * dados[[B]])) / (1.0 - 0.0722)))
  dados$MGRVI <- (dados[[G]]^2 - dados[[R]]^2) / (dados[[G]]^2 + dados[[R]]^2)
  dados$NB <- dados[[B]] / (dados[[R]] + dados[[G]] + dados[[B]])
  dados$NGBDI <- (dados[[G]] - dados[[B]]) / (dados[[G]] + dados[[B]])
  dados$NRBDI <- (dados[[R]] - dados[[B]]) / (dados[[R]] + dados[[B]])
  dados$NG <- dados[[G]] / (dados[[R]] + dados[[G]] + dados[[B]])
  dados$NGRDI <- (dados[[G]] - dados[[R]]) / (dados[[G]] + dados[[R]])
  dados$NR <- dados[[R]] / (dados[[R]] + dados[[G]] + dados[[B]])
  dados$RB <- dados[[R]] - dados[[B]]
  dados$RGBindex <- (dados[[G]]^2 - dados[[B]] * dados[[R]]) / (dados[[G]]^2 + dados[[B]] * dados[[R]])
  dados$RG <- dados[[R]] - dados[[G]]
  dados$RGRI <- dados[[R]] / dados[[G]]
  dados$S <- ((dados[[R]] + dados[[G]] + dados[[B]]) - 3 * dados[[B]]) / (dados[[R]] + dados[[G]] + dados
[[B]])
  dados$SCI <- (dados[[R]] - dados[[G]]) / (dados[[R]] + dados[[G]])
  dados$TGI <- (-0.5) * ((190) * (dados[[R]] - dados[[G]]) - (120) * (dados[[R]] - dados[[B]])) / (dados
[[R]] + dados[[G]] + dados[[B]])
  dados$VARI <- (dados[[G]] - dados[[R]]) / (dados[[G]] + dados[[R]] - dados[[B]])
  dados$VEG <- dados[[G]] / (dados[[G]] + dados[[R]] + dados[[B]]) / ((dados[[R]] / (dados[[R]] + dados[[G]]
+ dados[[B]]))^0.667) * ((dados[[B]] / (dados[[R]] + dados[[G]] + dados[[B]]))^(1 - 0.667))
```

```
  return(dados)
}
```

3 Dados

3.1 Como o banco simulado foi construído

- **Conjunto I (URNRS 23):** 464 parcelas × 30 fotos. Cada parcela tem um **estado de vigor foliar** (verdor), um nível de **senescência/doença** e um efeito de **local no campo**; o RGHA depende dos três, o vigor entra com **saturação** (`tanh` : acima de certo verdor a produtividade quase não sobe) e há muito efeito não explicado pela folha.
- **Foto:** vigor e senescência da foto = valor da parcela + variação entre folíolos; as bandas R, G, B saem de uma combinação linear dos dois, multiplicada por um fator de **iluminação** (celular, sombra) e com ruído. Cerca de 1,5% das fotos recebem **sombra ou flash** (artefato), para o passo de outliers ter o que remover.
- **Conjunto II:** 15 cultivares em **outro ambiente** (rendimento médio −300 kg ha^{-1}, vigor e luz ligeiramente diferentes), 2 repetições × 3 folíolos.
- **Conjunto III:** 215 linhagens F5, **mesmo ambiente do I**, 2 repetições × 3 folíolos, com mais variação genética que as cultivares.
- Como no Artigo, **todas as fotos de uma parcela compartilham o mesmo RGHA**: as linhas não são independentes (isso importa para a validação).

```
simular_artigo4 <- function(cfg) {
  set.seed(cfg$semente)
  bandas <- function(v, s, ilum) {
    n <- length(v)
    R <- ilum * (98 - 16 * v + 12 * s) + rnorm(n, 0, 3); G <- ilum * (128 + 14 * v - 10 * s) + rnorm(n, 0,
3); B <- ilum * (62 + 2 * v + 3 * s) + rnorm(n, 0, 2.5)
    art <- runif(n) < 0.015; f <- ifelse(runif(n) < 0.5, 0.55, 1.5)
    R[art] <- R[art] * f[art]; G[art] <- G[art] * f[art]; B[art] <- B[art] * f[art]
    data.frame(R = pmin(pmax(R, 1), 255), G = pmin(pmax(G, 1), 255), B = pmin(pmax(B, 1), 255))
  }
  rend <- function(v, s, micro, g, off = 0, sd_e) 3500 + off + 260 * v - 130 * (v^2 - 1) - 150 * s * plogis
(1.5 * v) + 150 * micro + g + rnorm(length(v), 0, sd_e)
  np <- cfg$n_parcelas_I
  micro <- as.numeric(scale(0.7 * sin(seq_len(np) / 25) + rnorm(np, 0, 0.7)))
  v <- 0.55 * micro + sqrt(1 - 0.55^2) * rnorm(np); s <- rnorm(np)
  yI <- rend(v, s, micro, 0, 0, 650)
  fI <- cfg$fotos_por_parcela; ip <- rep(seq_len(np), each = fI)
  I <- data.frame(conjunto = "I", unidade = sprintf("P%03d", ip), repeticao = NA_integer_, foliolo = rep(seq
_len(fI), np),
                  bandas(v[ip] + rnorm(np * fI, 0, 0.35), s[ip] + rnorm(np * fI, 0, 0.35), exp(rnorm(np * f
I, 0, 0.12))), RGHA = yI[ip])
  mk <- function(conj, nc, pref, sd_gv, sd_g, off, dv, ilum_m, sd_e) {
    gv <- rnorm(nc, 0, sd_gv); g <- sd_g * (0.4 * gv / sd_gv + sqrt(1 - 0.16) * rnorm(nc))
    gr <- expand.grid(cult = seq_len(nc), rep = seq_len(cfg$reps), fol = seq_len(cfg$foliolos))
    pl <- unique(gr[c("cult", "rep")]); n <- nrow(pl)
    vp <- gv[pl$cult] + dv + rnorm(n, 0, 0.5); sp <- rnorm(n); mp <- rnorm(n, 0, 0.5)
    yp <- rend(vp, sp, mp, g[pl$cult], off, sd_e)
    ix <- match(paste(gr$cult, gr$rep), paste(pl$cult, pl$rep)); m <- nrow(gr)
    data.frame(conjunto = conj, unidade = sprintf("%s%03d", pref, gr$cult), repeticao = gr$rep, foliolo = gr
$fol,
               bandas(vp[ix] + rnorm(m, 0, 0.35), sp[ix] + rnorm(m, 0, 0.35), ilum_m * exp(rnorm(m, 0, 0.1
2))), RGHA = yp[ix])
  }
  II  <- mk("II", cfg$n_cultivares_II, "C", 0.7, 300, -300, -0.15, 1.03, 450)
  III <- mk("III", cfg$n_linhagens_III, "L", 0.9, 330, 0, 0, 1.00, 450)
  d <- rbind(I, II, III); d$RGHA <- round(d$RGHA, 1); d[c("R", "G", "B")] <- lapply(d[c("R", "G", "B")], rou
nd, 1); d
}
```

3.2 Leitura do banco (Excel)

Se `dados_artigo4.xlsx` não existe, o script simula e grava o arquivo (abas `dados` e `dicionario`). O mesmo arquivo abre no R com `openxlsx::read.xlsx()` .

```
if (!file.exists(CFG$arquivo)) {
  sim <- simular_artigo4(CFG)
  dic <- data.frame(coluna = names(sim), descricao = c("conjunto: I (construção), II (testemunhas em outro a
mbiente) ou III (linhagens F5)",
    "parcela (conjunto I), cultivar (II) ou linhagem (III)", "repetição (II e III)", "folíolo fotografado",
"banda vermelha média do folíolo (0-255)",
    "banda verde média (0-255)", "banda azul média (0-255)", "rendimento de grãos da parcela (kg/ha); igual
para as fotos da mesma unidade e repetição"))
  openxlsx::write.xlsx(list(dados = sim, dicionario = dic), CFG$arquivo, overwrite = TRUE)
}
bruto <- openxlsx::read.xlsx(CFG$arquivo, sheet = "dados")
bruto[c("R", "G", "B", "RGHA")] <- lapply(bruto[c("R", "G", "B", "RGHA")], as.numeric)
bruto$parcela <- ifelse(bruto$conjunto == "I", bruto$unidade, paste(bruto$unidade, bruto$repeticao, sep = "_
r"))
res <- do.call(rbind, lapply(split(bruto, bruto$conjunto), function(x) data.frame(conjunto = x$conjunto[1],
fotos = nrow(x), unidades = length(unique(x$unidade)),
  parcelas = length(unique(x$parcela)), RGHA_medio = mean(x$RGHA), RGHA_dp = sd(x$RGHA), RGHA_min = min(x$RG
HA), RGHA_max = max(x$RGHA))))
mostra(res, "Conjuntos de dados. RGHA em kg ha⁻¹; parcelas = combinações de unidade e repetição que comparti
lham o mesmo RGHA.", d = 0)
```

Tabela 1. Conjuntos de dados. RGHA em kg ha⁻¹; parcelas = combinações de unidade e repetição que compartilham o mesmo RGHA.

conjunto	fotos	unidades	parcelas	RGHA_medio	RGHA_dp	RGHA_min	RGHA_max
I	13920	464	464	3555	754	1125	5374
II	90	15	30	3212	792	1123	5220
III	1290	215	430	3525	707	1139	5353

4 Pré-processamento

4.1 Outliers nas bandas

IQR 1,5 em cada banda (R, G, B), dentro de cada conjunto, para tirar fotos com sombra, flash ou segmentação ruim. O RGHA **não** entra no corte: rendimento extremo de uma parcela é dado real.

```
out_iqr <- function(x, k = CFG$iqr_k) { q <- quantile(x, c(.25, .75)); i <- diff(q); x < q[1] - k * i | x >
q[2] + k * i }
bruto$out <- ave(seq_len(nrow(bruto)), bruto$conjunto, FUN = function(i) as.integer(out_iqr(bruto$R[i]) | ou
t_iqr(bruto$G[i]) | out_iqr(bruto$B[i])))
tab_out <- do.call(rbind, lapply(split(bruto, bruto$conjunto), function(x) data.frame(conjunto = x$conjunto
[1], antes = nrow(x), removidas = sum(x$out), depois = sum(x$out == 0), perc_removido = 100 * mean(x$out))))
mostra(tab_out, "Outliers removidos pelo IQR 1,5 nas bandas R, G e B, por conjunto.", d = 1)
```

Tabela 2. Outliers removidos pelo IQR 1,5 nas bandas R, G e B, por conjunto.

conjunto	antes	removidas	depois	perc_removido
I	13920	423	13497	3.0
II	90	4	86	4.4
III	1290	51	1239	4.0

```
dados <- bruto[bruto$out == 0, ]; dados$out <- NULL
```

4.2 Extração dos índices

`Breed_RGB` sobre as bandas de cada foto. Índices não finitos (denominador ~ 0) ou constantes são descartados e listados.

```
dados <- as.data.frame(Breed_RGB(dados, R, G, B))
if (CFG$exgr == "diferenca") dados$ExGR <- dados$EXGRaw - dados$ExR
nomes_indices <- setdiff(names(dados), c("conjunto", "unidade", "repeticao", "foliolo", "R", "G", "B", "RGH
A", "parcela"))
ruim <- nomes_indices[!sapply(nomes_indices, function(v) all(is.finite(dados[[v]])) && sd(dados[[v]]) > 0)]
indices <- setdiff(nomes_indices, ruim)
tab_idx <- data.frame(etapa = c("índices calculados pelo Breed_RGB", "descartados (não finitos ou constante
s)", "índices candidatos"), n = c(length(nomes_indices), length(ruim), length(indices)),
                      detalhe = c("", paste(ruim, collapse = ", "), ""))
mostra(tab_idx, "Índices calculados e descartados antes da seleção.")
```

Tabela 3. Índices calculados e descartados antes da seleção.

etapa	n	detalhe
índices calculados pelo Breed_RGB	49	
descartados (não finitos ou constantes)	0	
índices candidatos	49	

5 Seleção dos índices: regressão ReliefF

Implementação própria do **RReliefF** (Robnik-Šikonja e Kononenko, 1997) com vizinhos de peso igual, para não depender de pacote. Roda só no **conjunto I de treino** (80%), sobre índices convertidos em postos (0-1), que neutralizam valores extremos. Como muitos índices são quase o mesmo (VARI, NGRDI, GLI…), a lista de candidatos que entra no modelo pode pular índices com $|r| > 0{,}95$ em relação a um já escolhido.

```
I <- dados[dados$conjunto == "I", ]
## partição 80/20 como no Artigo: por FOTOS (linhas), estratificada pelo RGHA
set.seed(CFG$semente)
i_tr <- as.vector(caret::createDataPartition(I$RGHA, p = 1 - CFG$prop_teste, list = FALSE)); I$grupo <- "val
idacao"; I$grupo[i_tr] <- "treino"
rrelief <- function(X, y, m = CFG$n_relief, k = CFG$k_relief) {
  X <- as.matrix(X); Xs <- apply(X, 2, function(x) rank(x) / length(x)); colnames(Xs) <- colnames(X); ys <-
(y - min(y)) / (max(y) - min(y))
  q <- sample(nrow(Xs), min(m, nrow(Xs))); nn <- FNN::get.knnx(Xs, Xs[q, , drop = FALSE], k = k + 1)$nn.inde
x[, -1, drop = FALSE]
  dy <- abs(ys[q] - matrix(ys[nn], nrow = length(q)))
  Pdc <- mean(dy); Pda <- Pdcda <- numeric(ncol(Xs))
  for (j in seq_len(ncol(Xs))) { da <- abs(Xs[q, j] - matrix(Xs[nn, j], nrow = length(q))); Pda[j] <- mean(d
a); Pdcda[j] <- mean(dy * da) }
  w <- Pdcda / Pdc - (Pda - Pdcda) / (1 - Pdc); names(w) <- colnames(Xs); w
}
w <- rrelief(I[I$grupo == "treino", indices], I$RGHA[I$grupo == "treino"])
rank_relief <- data.frame(posicao = seq_along(w), indice = names(sort(w, decreasing = TRUE)), ReliefF = sort
(w, decreasing = TRUE), row.names = NULL)
rank_relief$r_RGHA <- sapply(rank_relief$indice, function(v) cor(I[[v]][I$grupo == "treino"], I$RGHA[I$grupo
== "treino"]))
rank_relief$posicao_por_r <- rank(-abs(rank_relief$r_RGHA), ties.method = "min")
cm <- abs(cor(I[I$grupo == "treino", indices]))
esc <- c(); for (v in rank_relief$indice) if (length(esc) < CFG$n_indices && (!length(esc) || all(cm[v, esc]
<= CFG$colin_r_max))) esc <- c(esc, v)
usar <- if (is.null(CFG$indices_modelo)) esc else CFG$indices_modelo
rr_tab <- rbind(head(rank_relief, 12), rank_relief[rank_relief$indice %in% setdiff(CFG$indices_modelo, head
(rank_relief$indice, 12)), ]); rr_tab$ReliefF <- rr_tab$ReliefF * 1000; names(rr_tab)[3] <- "ReliefF_x1000"
mostra(rr_tab,
       "Regressão ReliefF: 12 melhores índices e a posição dos índices do Artigo 4 (VARI e ExGR) quando fora
dessa lista. Pontuação (×1000) maior = índice que muda junto com o RGHA; r_RGHA = correlação simples e posic
ao_por_r = posição pelo |r|.", d = 3)
```

Tabela 4. Regressão ReliefF: 12 melhores índices e a posição dos índices do Artigo 4 (VARI e ExGR) quando fora dessa lista. Pontuação (×1000) maior = índice que muda junto com o RGHA; r_RGHA = correlação simples e posicao_por_r = posição pelo |r|.

posicao	indice	ReliefF_x1000	r_RGHA	posicao_por_r
1	NB	0.268	0.174	42
2	S	0.268	-0.174	42
3	BGI	0.248	-0.292	36
4	IGB	0.248	0.297	34
5	NGBDI	0.248	0.295	35
6	bx	0.145	-0.240	37
7	ExB	0.134	-0.326	31
8	NRBDI	0.107	-0.399	8
9	IRB	0.107	-0.425	1
10	CI	0.107	-0.373	26
11	BG	0.107	-0.321	32
12	GB	0.107	0.321	32
28	ExGR	-0.040	0.386	19
39	VARI	-0.054	0.393	15

```
salva(rank_relief, "relief_ranking")
texto("**Cuidado ao ler o ReliefF:** ele procura vizinhos no espaço de todos os índices. Como cerca de 20 do
s 49 índices medem brilho (BI, L, I, GRAY...), eles dominam a distância e podem subir no ranking sem relação
com o RGHA; por isso o ranking é lido junto com a correlação simples. ",
      "**Índices que entram no modelo:** ", paste(usar, collapse = " e "), ". Escolha do ReliefF após filtro
de colinearidade: ", paste(esc, collapse = " e "),
      ". Para o ReliefF decidir sozinho, use `CFG$indices_modelo <- NULL`.")
```

Cuidado ao ler o ReliefF: ele procura vizinhos no espaço de todos os índices. Como cerca de 20 dos 49 índices medem brilho (BI, L, I, GRAY…), eles dominam a distância e podem subir no ranking sem relação com o RGHA; por isso o ranking é lido junto com a correlação simples. **Índices que entram no modelo:** VARI e ExGR. Escolha do ReliefF após filtro de colinearidade: NB e BGI. Para o ReliefF decidir sozinho, use `CFG$indices_modelo <- NULL`.

5.1 Colinearidade entre os índices escolhidos

Com dois índices derivados das mesmas três bandas, a correlação entre eles mostra o quanto a segunda informação é nova.

```
cc <- cor(I[I$grupo == "treino", usar]); vif <- diag(solve(cc))
mostra(data.frame(indice = usar, r_com_o_outro = if (length(usar) == 2) rep(cc[1, 2], 2) else NA, VIF = vif,
r_com_RGHA = sapply(usar, function(v) cor(I[[v]][I$grupo == "treino"], I$RGHA[I$grupo == "treino"]))),
       "Índices do modelo final: correlação entre eles, VIF e correlação com o RGHA (treino do conjunto
I).", d = 3)
```

Tabela 5. Índices do modelo final: correlação entre eles, VIF e correlação com o RGHA (treino do conjunto I).

indice	r_com_o_outro	VIF	r_com_RGHA
VARI	0.985	33.615	0.393
ExGR	0.985	33.615	0.386

6 Os 14 métodos

Todos usam as mesmas preditoras (`usar` , padronizadas pela média e desvio do treino) e o RGHA como resposta. `ajusta()` treina com um conjunto e prediz vários outros de uma vez. **`nLnn*`** : não consegui identificar esse método na lista do Artigo; entra como regressão polinomial de grau 2 (com interação) para marcar o lugar. Troque pelo seu em `ajusta()` . **Deep Learning**: perceptron de 2 camadas ocultas escrito em R puro (sem Keras), para o exemplo rodar em qualquer máquina; com Keras/TensorFlow instalado, substitua o bloco.

Tabela 6. Métodos comparados (13 neste computador): formulação e ajuste. Todos usam as mesmas preditoras padronizadas; a resposta é o RGHA (kg ha⁻¹).

Metodo	Formulacao	Ajuste
Lm	$\hat{y} = \beta 0 + \Sigma\, \beta j \cdot xj$, mínimos quadrados.	—
Ridge	Mínimos quadrados + $\lambda \cdot \Sigma \beta j^2$ (L2): encolhe coeficientes; estável com índices correlacionados.	λ por cv.glmnet (10 partes)
Bayesian Ridge	Regressão linear com prior normal nos coeficientes; precisão do erro e do prior estimadas pelos dados (maximização da evidência, como no BayesianRidge do scikit-learn).	automático
Lasso	Mínimos quadrados + $\lambda \cdot \Sigma \lvert \beta j \rvert$ (L1): zera coeficientes.	λ por cv.glmnet (10 partes)
nLnn*	Polinômio de grau 2 em cada índice mais a interação (substituto de nLnn).	—
GAM	$\hat{y} = \beta 0 + \Sigma\, fj(xj)$, com fj splines penalizadas: curvas suaves, sem interação.	k = 5 por termo, REML
Random Forest	Média de 500 árvores em amostras bootstrap; capta limiares e interações.	mtry = 1 ou 2 (ranger)
SVM	Regressão por vetores de suporte com kernel radial; só pontos fora da margem ε definem a superfície.	custo 1; γ = 1/p; ε = 0,1 (RGHA padronizado)
KNN	Média do RGHA dos k vizinhos mais próximos no espaço dos índices padronizados.	k de {5, 10, 20, 40, 80, 160} por CV de 5 partes
RNA	Rede neural com 1 camada oculta (5 neurônios, logística) e saída linear.	decay 0,01; 200 iterações
BRM	Regressão bayesiana (priors Cauchy fracos, pacote arm); a predição é a média posterior. Para brms use `brms::brm()`.	padrão do arm
Deep Learning	Perceptron com 2 camadas ocultas (16 e 8 neurônios, ReLU), Adam, 400 épocas.	lr 0,01; 400 épocas
Gradient Boosting	Boosting com o ajuste do Artigo 4: 100 árvores, taxa 0,10, profundidade 1, subamostra 1,0 (scikit-learn; aqui pacote gbm).	n.minobsinnode = 3 (aproxima o ‘nós dividem subconjuntos menores que 3’ do Artigo)

```
bayes_ridge <- function(X, y, iter = 200) {
  mx <- colMeans(X); yc <- y - mean(y); Xc <- sweep(X, 2, mx); n <- nrow(X); p <- ncol(X); XtX <- crossprod
(Xc); ev <- eigen(XtX, symmetric = TRUE)$values
  a <- 1 / var(yc); l <- 1
  for (i in seq_len(iter)) {
    S <- solve(a * XtX + l * diag(p)); w <- a * S %*% crossprod(Xc, yc); g <- sum(a * ev / (l + a * ev))
    l2 <- g / sum(w^2); a2 <- (n - g) / sum((yc - Xc %*% w)^2); if (abs(l2 - l) + abs(a2 - a) < 1e-8 * (l +
a)) break; l <- l2; a <- a2
  }
  function(Xn) as.numeric(mean(y) + sweep(Xn, 2, mx) %*% w)
}
mlp <- function(X, y, h = c(16, 8), epocas = 400, lr = 0.01, semente = 1) {
  set.seed(semente); ym <- mean(y); ys <- sd(y); yy <- (y - ym) / ys; tam <- c(ncol(X), h, 1); L <- length(t
am) - 1
  W <- lapply(seq_len(L), function(l) matrix(rnorm(tam[l] * tam[l + 1], 0, sqrt(2 / tam[l])), tam[l])); b <-
lapply(seq_len(L), function(l) rep(0, tam[l + 1]))
  mW <- lapply(W, function(w) w * 0); vW <- mW; mb <- lapply(b, function(x) x * 0); vb <- mb
  fwd <- function(X) { A <- list(X); for (l in seq_len(L)) { Z <- sweep(A[[l]] %*% W[[l]], 2, b[[l]], "+");
A[[l + 1]] <- if (l < L) pmax(Z, 0) else Z }; A }
  n <- nrow(X)
  for (e in seq_len(epocas)) {
    A <- fwd(X); d <- (A[[L + 1]] - yy) * 2 / n
    for (l in L:1) {
      gW <- crossprod(A[[l]], d); gb <- colSums(d); if (l > 1) d <- (d %*% t(W[[l]])) * (A[[l]] > 0)
      mW[[l]] <- 0.9 * mW[[l]] + 0.1 * gW; vW[[l]] <- 0.999 * vW[[l]] + 0.001 * gW^2; mb[[l]] <- 0.9 * mb
[[l]] + 0.1 * gb; vb[[l]] <- 0.999 * vb[[l]] + 0.001 * gb^2
      W[[l]] <- W[[l]] - lr * (mW[[l]] / (1 - 0.9^e)) / (sqrt(vW[[l]] / (1 - 0.999^e)) + 1e-8); b[[l]] <- b
[[l]] - lr * (mb[[l]] / (1 - 0.9^e)) / (sqrt(vb[[l]] / (1 - 0.999^e)) + 1e-8)
    }
  }
  function(Xn) as.numeric(fwd(Xn)[[L + 1]]) * ys + ym
}
ajusta <- function(tr, novos, vars, modelos = METODOS, resposta = "RGHA", semente = CFG$semente) {
  mu <- sapply(tr[vars], mean); dp <- sapply(tr[vars], sd)
  z <- function(d) { d <- as.data.frame(d[vars]); d[] <- Map(function(v, m, s) (v - m) / s, d, mu, dp); d }
  trz <- z(tr); y <- tr[[resposta]]; ym <- mean(y); ysd <- sd(y); yz <- (y - ym) / ysd
  Xtr <- as.matrix(trz); trz[[resposta]] <- y; f <- reformulate(vars, resposta); saida <- list(); falhas <-
c()
  for (m in intersect(modelos, METODOS)) {
    set.seed(semente)
    pr <- tryCatch(switch(m,
      "Lm" = { a <- lm(f, trz); function(d) as.numeric(predict(a, d)) },
      "Ridge" = { a <- glmnet::glmnet(Xtr, y, alpha = 0, lambda = glmnet::cv.glmnet(Xtr, y, alpha = 0)$lambd
a.min); function(d) as.numeric(predict(a, as.matrix(d))) },
      "Bayesian Ridge" = { a <- bayes_ridge(Xtr, y); function(d) a(as.matrix(d)) },
      "Lasso" = { a <- glmnet::glmnet(Xtr, y, alpha = 1, lambda = glmnet::cv.glmnet(Xtr, y, alpha = 1)$lambd
a.min); function(d) as.numeric(predict(a, as.matrix(d))) },
      "nLnn*" = { ff <- reformulate(c(paste0("poly(", vars, ", 2)"), if (length(vars) == 2) paste(vars, coll
apse = ":")), resposta); a <- lm(ff, trz); function(d) as.numeric(predict(a, d)) },
      "GAM" = { a <- mgcv::gam(reformulate(paste0("s(", vars, ", k = 5)"), resposta), data = trz, method =
"REML"); function(d) as.numeric(predict(a, d)) },
      "Random Forest" = { a <- ranger::ranger(f, trz, num.trees = 500, mtry = max(1, floor(sqrt(length(var
s)))), seed = semente); function(d) predict(a, d)$predictions },
      "XGBoost" = { a <- xgboost::xgb.train(params = list(objective = "reg:squarederror", max_depth = 4, eta
= 0.1, nthread = 1), data = xgboost::xgb.DMatrix(Xtr, label = y), nrounds = 200, verbose = 0)
        function(d) predict(a, xgboost::xgb.DMatrix(as.matrix(d))) },
      "SVM" = { a <- e1071::svm(Xtr, yz, type = "eps-regression", kernel = "radial", cost = 1, gamma = 1 / n
col(Xtr), epsilon = 0.1); function(d) as.numeric(predict(a, as.matrix(d))) * ysd + ym },
      "KNN" = { ks <- c(5, 10, 20, 40, 80, 160); fo <- caret::createFolds(y, k = 5)
        err <- sapply(ks, function(k) mean(sapply(fo, function(i) rmse(y[i], FNN::knn.reg(Xtr[-i, , drop = F
ALSE], Xtr[i, , drop = FALSE], y[-i], k = k)$pred))))
        k <- ks[which.min(err)]; function(d) FNN::knn.reg(Xtr, as.matrix(d), y, k = k)$pred },
      "RNA" = { a <- nnet::nnet(Xtr, yz, size = 5, decay = 0.01, linout = TRUE, maxit = 200, trace = FALSE);
function(d) as.numeric(predict(a, as.matrix(d))) * ysd + ym },
      "BRM" = { a <- arm::bayesglm(f, data = trz, family = gaussian()); function(d) as.numeric(predict(a,
d)) },
      "Deep Learning" = { a <- mlp(Xtr, y, semente = semente); function(d) a(as.matrix(d)) },
```

```
      "Gradient Boosting" = { g <- CFG$gb
        a <- gbm::gbm(f, data = trz, distribution = "gaussian", n.trees = g$arvores, shrinkage = g$taxa, int
eraction.depth = g$profundidade, bag.fraction = g$subamostra,
                      n.minobsinnode = g$min_obs_no, train.fraction = 1, verbose = FALSE, n.cores = 1)
        function(d) as.numeric(predict(a, d, n.trees = g$arvores)) }),
      error = function(e) { falhas <<- c(falhas, paste0(m, ": ", conditionMessage(e))); NULL })
    if (!is.null(pr)) saida[[m]] <- c(list(tr = pr(trz[vars])), lapply(novos, function(d) pr(z(d))))
  }
  attr(saida, "falhas") <- falhas; saida
}
```

7 Treinamento (80%) e avaliação no conjunto de validação (20%)

Partição 80/20 **estratificada pelo RGHA** (quantis), como no Artigo. As fotos são sorteadas individualmente, então fotos da mesma parcela caem nos dois lados: é a forma usada no Artigo e a mais otimista (veja o diagnóstico de independência mais adiante). **R² treino − R² validação** mede sobreajuste.

```
TR <- I[I$grupo == "treino", ]; VA <- I[I$grupo == "validacao", ]
II_ <- dados[dados$conjunto == "II", ]; III_ <- dados[dados$conjunto == "III", ]
tempo <- system.time(ajuste <- ajusta(TR, list(val = VA, II = II_, III = III_), usar))
if (length(attr(ajuste, "falhas"))) stop("Falharam: ", paste(attr(ajuste, "falhas"), collapse = "; "))
tab_val <- do.call(rbind, lapply(names(ajuste), function(m) cbind(Metodo = m, metricas(VA$RGHA, ajuste[[m]]
$val),
  R2_treino = r2(TR$RGHA, ajuste[[m]]$tr), RMSE_treino = rmse(TR$RGHA, ajuste[[m]]$tr))))
tab_val$gap_R2 <- tab_val$R2_treino - tab_val$R2
tab_val <- tab_val[order(tab_val$RMSE), ]
mostra(tab_val, paste0("Desempenho no conjunto de validação (20% do conjunto I; n = ", nrow(VA), " fotos), o
rdenado pelo RMSE. Colunas _treino: desempenho nas próprias fotos de treino (n = ", nrow(TR), "); gap_R2 = s
obreajuste. RMSE e MAE em kg ha⁻¹; MAPE em %."), d = 3)
```

Tabela 7. Desempenho no conjunto de validação (20% do conjunto I; n = 2697 fotos), ordenado pelo RMSE. Colunas _treino: desempenho nas próprias fotos de treino (n = 10800); gap_R2 = sobreajuste. RMSE e MAE em kg ha⁻¹; MAPE em %.

Metodo	R2	RMSE	MAE	MAPE	R2_treino	RMSE_treino	gap_R2
KNN	0.152	691.043	554.709	17.333	0.173	683.443	0.021
RNA	0.150	691.539	552.642	17.246	0.175	682.809	0.024
GAM	0.150	691.545	554.733	17.302	0.169	685.230	0.018
Deep Learning	0.149	692.037	554.584	17.304	0.171	684.159	0.022
Gradient Boosting	0.148	692.601	555.415	17.345	0.165	686.851	0.017
SVM	0.147	692.832	553.614	17.336	0.170	684.831	0.022
nLnn*	0.147	692.864	555.590	17.336	0.164	687.240	0.017
Ridge	0.145	693.938	556.432	17.399	0.154	691.341	0.009
Lasso	0.143	694.381	556.554	17.393	0.155	690.968	0.011
Bayesian Ridge	0.143	694.611	556.698	17.393	0.155	690.934	0.012
BRM	0.143	694.688	556.761	17.394	0.155	690.932	0.012
Lm	0.143	694.689	556.762	17.394	0.155	690.932	0.012
Random Forest	0.032	738.060	589.143	18.296	0.782	350.497	0.750

```
salva(tab_val, "validacao_20pct")
melhor <- tab_val$Metodo[1]

pl <- do.call(rbind, lapply(c(melhor, "Lm", "Random Forest"), function(m) data.frame(metodo = m, obs = VA$RG
HA, pred = ajuste[[m]]$val)))
pl$metodo <- factor(pl$metodo, unique(c(melhor, "Lm", "Random Forest")))
mostra_fig(ggplot(pl, aes(pred, obs)) + geom_point(alpha = .08, size = .7, colour = "#2e7d32") + geom_abline
(colour = "#b71c1c") + geom_smooth(method = "lm", se = FALSE, formula = y ~ x, colour = "black", linewidth =
.6) +
  facet_wrap(~metodo) + labs(x = "RGHA predito (kg/ha)", y = "RGHA observado (kg/ha)") + theme_minimal(base_
size = 11),
  "fig1_obs_pred.png", "RGHA observado × predito no conjunto de validação (20%). Linha vermelha: 1:1; preta:
ajuste observado ~ predito. Predições encurtadas (reta mais plana que 1:1) são típicas de R² baixo.", 9, 3.
6)
```

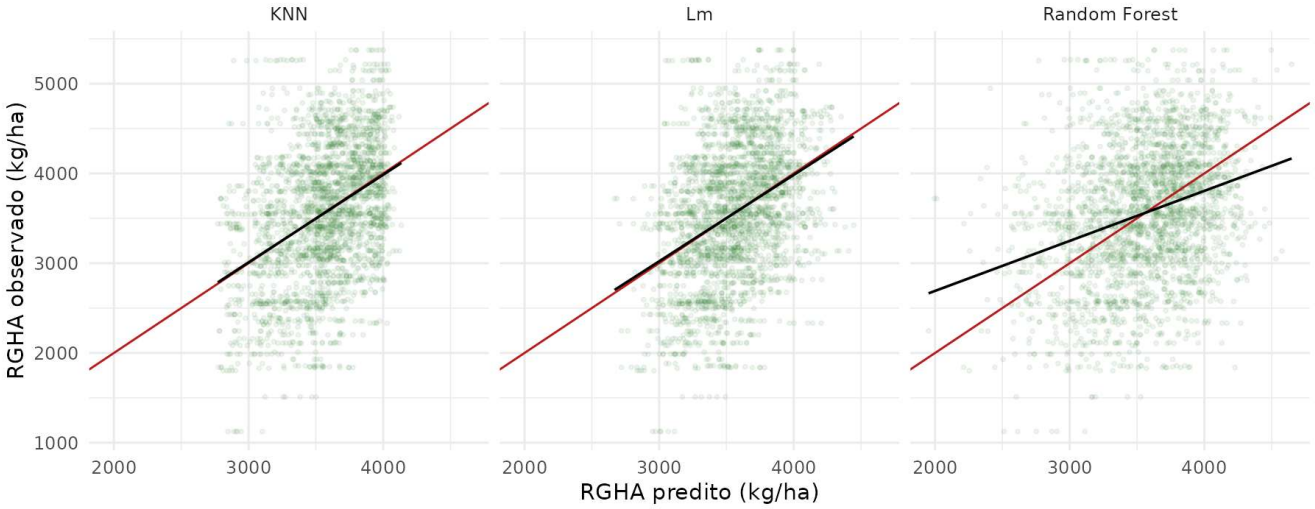


Figura 1. RGHA observado × predito no conjunto de validação (20%). Linha vermelha: 1:1; preta: ajuste observado ~ predito. Predições encurtadas (reta mais plana que 1:1) são típicas de R² baixo.

7.1 Validação cruzada de 20 partes estratificadas (dentro dos 80% de treino)

Cada parte (*fold*) fica de fora uma vez; a tabela traz média e desvio padrão das métricas nas partes. Desvio pequeno = método estável.

```
cv_metodos <- function(dd, folds, modelos = METODOS) {
  do.call(rbind, lapply(sort(unique(folds)), function(k) { a <- ajusta(dd[folds != k, ], list(te = dd[folds
== k, ]), usar, modelos)
    do.call(rbind, lapply(names(a), function(m) cbind(parte = k, Metodo = m, metricas(dd$RGHA[folds == k], a
[[m]]$te)))) }))
}
resume <- function(long) { r <- aggregate(cbind(R2, RMSE, MAE, MAPE) ~ Metodo, long, function(x) c(media = m
ean(x), dp = sd(x)))
  r <- do.call(data.frame, r); names(r) <- sub("\\.media", "", sub("\\.dp", "_dp", names(r))); r[order(r$RMS
E), ] }
set.seed(CFG$semente); cv_long <- cv_metodos(TR, caret::createFolds(TR$RGHA, k = CFG$n_folds, list = FALSE))
tab_cv <- resume(cv_long)
mostra(tab_cv, paste0("Validação cruzada com ", CFG$n_folds, " partes estratificadas no treino: média das pa
rtes e desvio padrão (_dp), ordenado pelo RMSE."), d = 3); salva(tab_cv, "cv_partes")
```

Tabela 8. Validação cruzada com 20 partes estratificadas no treino: média das partes e desvio padrão (_dp), ordenado pelo RMSE.

Metodo	R2	R2_dp	RMSE	RMSE_dp	MAE	MAE_dp	MAPE	MAPE_dp
RNA	0.170	0.028	684.219	16.268	549.758	12.519	17.232	0.527
Deep Learning	0.168	0.028	685.411	15.916	551.549	12.390	17.294	0.523

Metodo	R2	R2_dp	RMSE	RMSE_dp	MAE	MAE_dp	MAPE	MAPE_dp
GAM	0.167	0.027	685.602	15.638	551.110	12.484	17.283	0.524
SVM	0.167	0.029	685.774	16.302	549.166	13.169	17.286	0.543
nLnn*	0.163	0.029	687.433	15.605	552.761	12.004	17.342	0.502
KNN	0.162	0.028	687.767	16.141	553.285	11.924	17.390	0.519
Gradient Boosting	0.156	0.027	690.102	14.776	554.620	10.999	17.420	0.507
Lasso	0.154	0.028	690.864	15.603	554.094	11.831	17.427	0.532
BRM	0.154	0.028	690.887	15.734	554.123	11.996	17.424	0.535
Lm	0.154	0.028	690.887	15.735	554.123	11.997	17.424	0.535
Bayesian Ridge	0.154	0.028	690.890	15.706	554.119	11.957	17.425	0.534
Ridge	0.153	0.027	691.257	15.302	554.396	11.416	17.444	0.527
Random Forest	0.063	0.045	726.914	20.888	584.436	16.362	18.199	0.600

7.2 10 repetições de treino 80% e teste 20%

A partição inteira é refeita com sementes diferentes; mostra o quanto o resultado depende de qual 20% sai.

```
rep_long <- do.call(rbind, lapply(seq_len(CFG$n_rep), function(r) { set.seed(CFG$semente + r); it <- as.vect
or(caret::createDataPartition(I$RGHA, p = 0.8, list = FALSE))
  a <- ajusta(I[it, ], list(te = I[-it, ]), usar); do.call(rbind, lapply(names(a), function(m) cbind(repetic
ao = r, Metodo = m, metricas(I$RGHA[-it], a[[m]]$te)))) }))
tab_rep <- resume(rep_long)
venceu <- table(factor(sapply(split(rep_long, rep_long$repeticao), function(x) x$Metodo[which.min(x$RMSE)]),
tab_rep$Metodo)) / CFG$n_rep * 100
tab_rep$vitorias_RMSE_perc <- as.numeric(venceu[tab_rep$Metodo])
mostra(tab_rep, paste0(CFG$n_rep, " repetições de 80% treino e 20% teste: média, desvio padrão (_dp) e % de
repetições em que o método teve o menor RMSE."), d = 3); salva(tab_rep, "repeticoes_80_20")
```

Tabela 9. 10 repetições de 80% treino e 20% teste: média, desvio padrão (_dp) e % de repetições em que o método teve o menor RMSE.

Metodo	R2	R2_dp	RMSE	RMSE_dp	MAE	MAE_dp	MAPE	MAPE_dp	vitorias_RMSE_perc
RNA	0.171	0.010	684.623	3.440	549.275	2.787	17.199	0.151	90
GAM	0.166	0.009	686.410	2.944	551.109	2.571	17.267	0.155	10
SVM	0.166	0.010	686.483	3.475	549.406	3.209	17.271	0.142	0
Deep Learning	0.166	0.009	686.729	3.312	551.815	2.700	17.290	0.146	0
KNN	0.162	0.009	688.016	3.239	552.903	3.281	17.362	0.153	0
nLnn*	0.161	0.009	688.511	3.259	553.071	2.795	17.337	0.154	0
Gradient Boosting	0.157	0.009	690.192	3.224	554.199	3.169	17.396	0.142	0
Lasso	0.153	0.010	691.758	3.643	554.422	3.457	17.419	0.147	0
Bayesian Ridge	0.153	0.010	691.780	3.635	554.404	3.444	17.417	0.147	0
BRM	0.153	0.010	691.781	3.642	554.407	3.451	17.416	0.147	0
Lm	0.153	0.010	691.781	3.642	554.407	3.451	17.416	0.147	0
Ridge	0.153	0.009	691.989	3.572	554.625	3.405	17.433	0.148	0
Random Forest	0.058	0.018	729.730	5.018	586.724	4.458	18.270	0.196	0

```
mostra_fig(ggplot(rep_long, aes(reorder(Metodo, RMSE, median), RMSE)) + geom_boxplot(fill = "#a5d6a7", outli
er.size = .8) + coord_flip() +
  labs(x = NULL, y = "RMSE no teste (kg/ha)") + theme_minimal(base_size = 11),
  "fig2_rmse_repeticoes.png", paste0("RMSE nas ", CFG$n_rep, " repetições de 80/20, por método. Caixa pequen
a = resultado estável entre partições."), 8, 4.2)
```

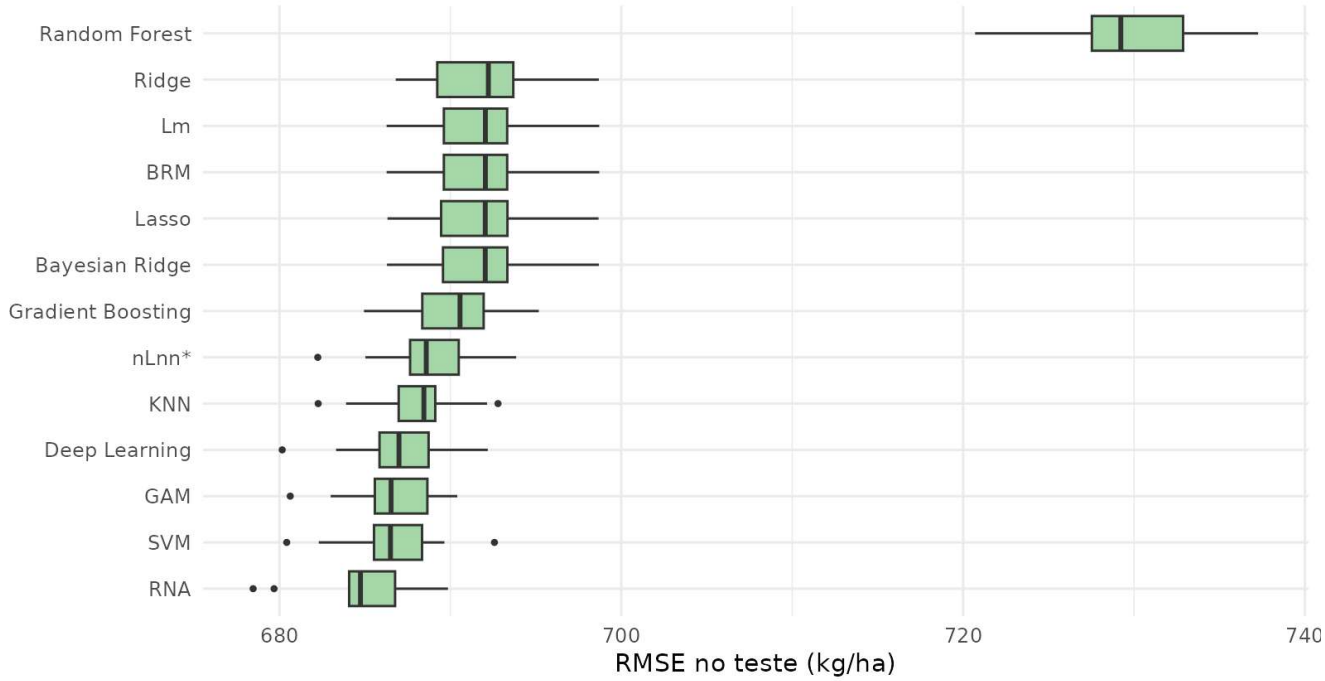


Figura 2. RMSE nas 10 repetições de 80/20, por método. Caixa pequena = resultado estável entre partições.

7.3 Método selecionado

Critério de desempenho: RMSE médio nas repetições (em kg ha⁻¹, mais interpretável que o R², aqui baixo). Como as diferenças entre os melhores métodos costumam ser menores que o desvio entre repetições, a tabela marca os que **empatam** com o melhor (RMSE médio até 1 desvio padrão acima do menor). Entre os empatados, a escolha segue critérios fora do erro: estabilidade, regularização e simplicidade. O método final do Artigo 4 (`CFG$metodo_final`) é o **Gradient Boosting**. R² baixo não invalida o método: reflete a variância do RGHA que a cor do folíolo não explica; por isso o artigo reporta RMSE, MAE e MAPE junto.

```
top <- tab_rep[1, ]; sel <- tab_rep[tab_rep$Metodo == CFG$metodo_final, ]
tab_rep$empata_com_o_melhor <- tab_rep$RMSE <= top$RMSE + top$RMSE_dp
mostra(tab_rep[c("Metodo", "RMSE", "RMSE_dp", "R2", "empata_com_o_melhor")], paste0("Empate técnico: método
com menor RMSE médio = ", top$Metodo, " (", round(top$RMSE, 1), " ± ", round(top$RMSE_dp, 1), " kg ha⁻¹). TR
UE = RMSE médio até 1 desvio padrão acima do menor. Método final: ", CFG$metodo_final, "."), d = 2)
```

Tabela 10. Empate técnico: método com menor RMSE médio = RNA (684.6 ± 3.4 kg ha⁻¹). TRUE = RMSE médio até 1 desvio padrão acima do menor. Método final: Gradient Boosting.

Metodo	RMSE	RMSE_dp	R2	empata_com_o_melhor
RNA	684.62	3.44	0.17	TRUE
GAM	686.41	2.94	0.17	TRUE
SVM	686.48	3.48	0.17	TRUE
Deep Learning	686.73	3.31	0.17	TRUE
KNN	688.02	3.24	0.16	TRUE
nLnn*	688.51	3.26	0.16	FALSE
Gradient Boosting	690.19	3.22	0.16	FALSE
Lasso	691.76	3.64	0.15	FALSE
Bayesian Ridge	691.78	3.63	0.15	FALSE

Metodo	RMSE	RMSE_dp	R2	empata_com_o_melhor
BRM	691.78	3.64	0.15	FALSE
Lm	691.78	3.64	0.15	FALSE
Ridge	691.99	3.57	0.15	FALSE
Random Forest	729.73	5.02	0.06	FALSE

8 Aplicação nos conjuntos externos

Os métodos treinados nos 80% do conjunto **I não são reajustados.** Conjunto II: outras cultivares, outro ambiente (testa transferência). Conjunto III: linhagens F5 no mesmo ambiente (testa uso na seleção). Como cada parcela tem 6 fotos com o mesmo RGHA, as métricas por foto repetem a mesma unidade; por isso há também a leitura **por unidade** (média das fotos) e o ranking.

```
ext <- function(d, nome) do.call(rbind, lapply(names(ajuste), function(m) { ve <- mean(ajuste[[m]][[nome]] - d$RGHA); data.frame(Metodo = m, metricas(d$RGHA, ajuste[[m]][[nome]]), vies = ve, RMSE_sem_vies = sqrt(max(rmse(d$RGHA, ajuste[[m]][[nome]])^2 - ve^2, 0))) }))
base <- setNames(tab_val$RMSE, tab_val$Metodo)
e2 <- ext(II_, "II"); e3 <- ext(III_, "III")
for (x in c("e2", "e3")) { t <- get(x); t$RMSE_validacao_20 <- base[t$Metodo]; t$razao_RMSE <- t$RMSE / t$RMSE_validacao_20; assign(x, t[order(t$RMSE), ]) }
mostra(e2, paste0("Conjunto II (", length(unique(II_$unidade)), " cultivares, outro ambiente; n = ", nrow(II_), " fotos). vies = predito − observado (kg ha⁻¹); RMSE_sem_vies = RMSE se o viés médio fosse corrigido (recalibração do ambiente); razao_RMSE = RMSE externo / RMSE da validação 20% (≈ 1 = desempenho mantido)."), d = 3); salva(e2, "externo_II")
```

Tabela 11. Conjunto II (15 cultivares, outro ambiente; n = 86 fotos). vies = predito − observado (kg ha⁻¹); RMSE_sem_vies = RMSE se o viés médio fosse corrigido (recalibração do ambiente); razao_RMSE = RMSE externo / RMSE da validação 20% (≈ 1 = desempenho mantido).

Metodo	R2	RMSE	MAE	MAPE	vies	RMSE_sem_vies	RMSE_validacao_20	razao_RMSE
RNA	0.099	760.966	616.065	24.304	290.742	703.234	691.539	1.100
KNN	0.085	766.963	626.609	24.508	283.789	712.528	691.043	1.110
Deep Learning	0.076	770.621	623.600	24.733	296.915	711.125	692.037	1.114
SVM	0.065	775.080	637.445	24.902	312.394	709.337	692.832	1.119
Gradient Boosting	0.055	779.475	639.602	25.006	282.681	726.411	692.601	1.125
GAM	0.054	779.746	634.129	25.118	297.287	720.850	691.545	1.128
nLnn*	0.047	782.719	633.067	25.167	289.258	727.309	692.864	1.130
BRM	0.043	784.440	642.931	25.247	285.618	730.595	694.688	1.129
Lm	0.043	784.440	642.931	25.247	285.619	730.594	694.689	1.129
Bayesian Ridge	0.043	784.499	643.064	25.249	285.701	730.625	694.611	1.129
Lasso	0.042	784.772	643.503	25.262	286.096	730.764	694.381	1.130
Ridge	0.039	785.859	645.020	25.307	287.384	731.426	693.938	1.132
Random Forest	0.025	791.728	633.399	24.592	289.816	736.777	738.060	1.073

```
mostra(e3, paste0("Conjunto III (", length(unique(III_$unidade)), " linhagens F5, mesmo ambiente; n = ", nrow(III_), " fotos), mesmas colunas da Tabela anterior."), d = 3); salva(e3, "externo_III")
```

Tabela 12. Conjunto III (215 linhagens F5, mesmo ambiente; n = 1239 fotos), mesmas colunas da Tabela anterior.

Metodo	R2	RMSE	MAE	MAPE	vies	RMSE_sem_vies	RMSE_validacao_20	razao_RMSE
RNA	0.238	614.707	484.312	15.613	24.815	614.206	691.539	0.889

Metodo	R2	RMSE	MAE	MAPE	vies	RMSE_sem_vies	RMSE_validacao_20	razao_RMSE
SVM	0.226	619.777	489.516	15.751	41.258	618.402	692.832	0.895
Deep Learning	0.225	620.043	489.352	15.759	24.431	619.562	692.037	0.896
GAM	0.223	621.032	490.859	15.775	21.777	620.650	691.545	0.898
KNN	0.218	622.738	491.685	15.873	25.199	622.228	691.043	0.901
nLnn*	0.216	623.647	492.866	15.840	21.188	623.287	692.864	0.900
Gradient Boosting	0.209	626.510	494.268	15.918	24.569	626.028	692.601	0.905
BRM	0.195	631.874	497.053	16.060	26.136	631.333	694.688	0.910
Lm	0.195	631.874	497.053	16.060	26.137	631.333	694.689	0.910
Bayesian Ridge	0.195	631.875	497.024	16.061	26.246	631.330	694.611	0.910
Lasso	0.195	631.922	496.948	16.064	26.613	631.361	694.381	0.910
Ridge	0.194	632.363	497.076	16.084	27.750	631.754	693.938	0.911
Random Forest	0.120	660.856	520.452	16.545	20.313	660.544	738.060	0.895

```
rank_unid <- function(d, nome) do.call(rbind, lapply(names(ajuste), function(m) {
  p <- tapply(ajuste[[m]][[nome]], d$unidade, mean); o <- tapply(d$RGHA, d$unidade, mean)[names(p)]; n20 <-
ceiling(0.2 * length(p))
  data.frame(Metodo = m, r_unidade = cor(p, o), spearman = cor(p, o, method = "spearman"), top20_acertos = s
um(names(sort(p, TRUE))[1:n20] %in% names(sort(o, TRUE))[1:n20]),
             top20_ao_acaso = n20^2 / length(p), n_selecionadas = n20) }))
r2_ <- rank_unid(II_, "II"); r3_ <- rank_unid(III_, "III")
mostra(r3_[order(-r3_$spearman), ], "Conjunto III, por linhagem (média das 6 fotos): correlação (r) e Spearm
an entre RGHA predito e observado e acertos na seleção do top 20% (top20_ao_acaso = acertos esperados sortea
ndo).", d = 2); salva(r3_, "ranking_linhagens")
```

Tabela 13. Conjunto III, por linhagem (média das 6 fotos): correlação (r) e Spearman entre RGHA predito e observado e acertos na seleção do top 20% (top20_ao_acaso = acertos esperados sorteando).

Metodo	r_unidade	spearman	top20_acertos	top20_ao_acaso	n_selecionadas
RNA	0.59	0.55	17	8.6	43
Deep Learning	0.57	0.54	17	8.6	43
SVM	0.57	0.53	17	8.6	43
GAM	0.57	0.53	16	8.6	43
KNN	0.56	0.53	17	8.6	43
nLnn*	0.56	0.52	16	8.6	43
Random Forest	0.54	0.51	15	8.6	43
Gradient Boosting	0.55	0.50	17	8.6	43
Lm	0.53	0.50	17	8.6	43
BRM	0.53	0.50	17	8.6	43
Bayesian Ridge	0.53	0.50	17	8.6	43
Lasso	0.53	0.50	17	8.6	43
Ridge	0.53	0.50	17	8.6	43

```
mostra(r2_[order(-r2_$spearman), ], "Conjunto II, por cultivar: mesmas colunas da tabela anterior (n pequen
o: leia com cautela).", d = 2)
```

Tabela 14. Conjunto II, por cultivar: mesmas colunas da tabela anterior (n pequeno: leia com cautela).

Metodo	r_unidade	spearman	top20_acertos	top20_ao_acaso	n_selecionadas
Random Forest	0.59	0.41	1	0.6	3
RNA	0.62	0.37	2	0.6	3
Deep Learning	0.60	0.36	2	0.6	3
Lm	0.56	0.34	2	0.6	3
Ridge	0.56	0.34	2	0.6	3
Bayesian Ridge	0.56	0.34	2	0.6	3
Lasso	0.56	0.34	2	0.6	3
nLnn*	0.57	0.34	2	0.6	3
BRM	0.56	0.34	2	0.6	3
KNN	0.59	0.33	2	0.6	3
SVM	0.60	0.32	2	0.6	3
GAM	0.57	0.32	2	0.6	3
Gradient Boosting	0.58	0.31	2	0.6	3

```
pe <- rbind(data.frame(conjunto = "II (outro ambiente)", obs = II_$RGHA, pred = ajuste[[sel$Metodo]]$II), da
ta.frame(conjunto = "III (linhagens F5)", obs = III_$RGHA, pred = ajuste[[sel$Metodo]]$III))
mostra_fig(ggplot(pe, aes(pred, obs)) + geom_point(alpha = .3, size = .9, colour = "#1565c0") + geom_abline
(colour = "#b71c1c") + geom_smooth(method = "lm", se = FALSE, formula = y ~ x, colour = "black", linewidth =
.6) +
  facet_wrap(~conjunto) + labs(x = "RGHA predito (kg/ha)", y = "RGHA observado (kg/ha)", title = paste("Méto
do selecionado:", sel$Metodo)) + theme_minimal(base_size = 11),
  "fig3_externos.png", "RGHA observado × predito nos conjuntos externos para o método selecionado. Linha ver
melha: 1:1; preta: ajuste observado ~ predito.", 9, 3.8)
```

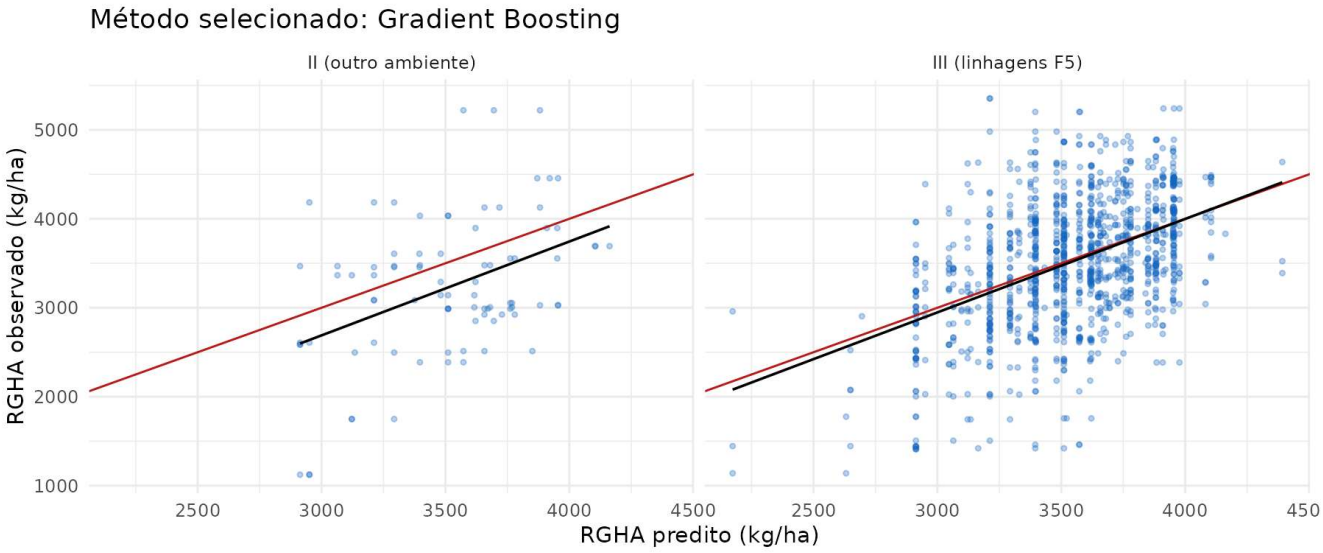


Figura 3. RGHA observado × predito nos conjuntos externos para o método selecionado. Linha vermelha: 1:1; preta: ajuste observado ~ predito.

9 Por que alguns métodos se adaptam melhor?

Quatro diagnósticos, cada um testa uma explicação. O texto de cada um é **hipótese a confrontar com a tabela**; a leitura automática no final resume os números desta execução.

9.1 1. A relação entre índice e RGHA é curva?

Se a resposta tem curvatura (saturação, ótimo), métodos que só ajustam retas (Lm, Ridge, Lasso, Bayesian Ridge, BRM) ficam atrás dos que ajustam curvas (GAM, boosting, SVM, redes). A figura mostra, para cada índice, o RGHA médio observado por faixa do índice (pontos) e o RGHA médio **predito** por cada método nas mesmas faixas (linhas), nas fotos de treino. Como VARI

e ExGR são quase o mesmo sinal (r próximo de 1), a curva é lida ao longo dos dados reais, sem fixar um índice e variar o outro (isso criaria combinações que não existem).

```
met_curva <- intersect(c("Lm", "GAM", "Random Forest", "KNN", "Deep Learning", "Gradient Boosting"), METODOS)
faixa <- function(v) cut(TR[[v]], unique(quantile(TR[[v]], seq(0, 1, .05))), include.lowest = TRUE)
cur <- do.call(rbind, lapply(usar, function(v) do.call(rbind, lapply(met_curva, function(m) data.frame(indice = v, metodo = m, x = as.numeric(tapply(TR[[v]], faixa(v), mean)), y = as.numeric(tapply(ajuste[[m]]$tr, faixa(v), mean)))))))
bin <- do.call(rbind, lapply(usar, function(v) data.frame(indice = v, x = as.numeric(tapply(TR[[v]], faixa(v), mean)), y = as.numeric(tapply(TR$RGHA, faixa(v), mean)))))
mostra_fig(ggplot() + geom_line(data = cur, aes(x, y, colour = metodo), linewidth = .8) + geom_point(data = bin, aes(x, y), colour = "grey20", size = 1.8) + facet_wrap(~indice, scales = "free_x") +
  labs(x = "valor do índice (média da faixa)", y = "RGHA (kg/ha)", colour = NULL) + theme_minimal(base_size = 11) + theme(legend.position = "bottom"),
  "fig4_curvas.png", "RGHA médio observado (pontos) e predito por método (linhas) em 20 faixas de cada índice, nas fotos de treino. Reta × curva mostra o que cada família consegue representar.", 9, 4.2)
```

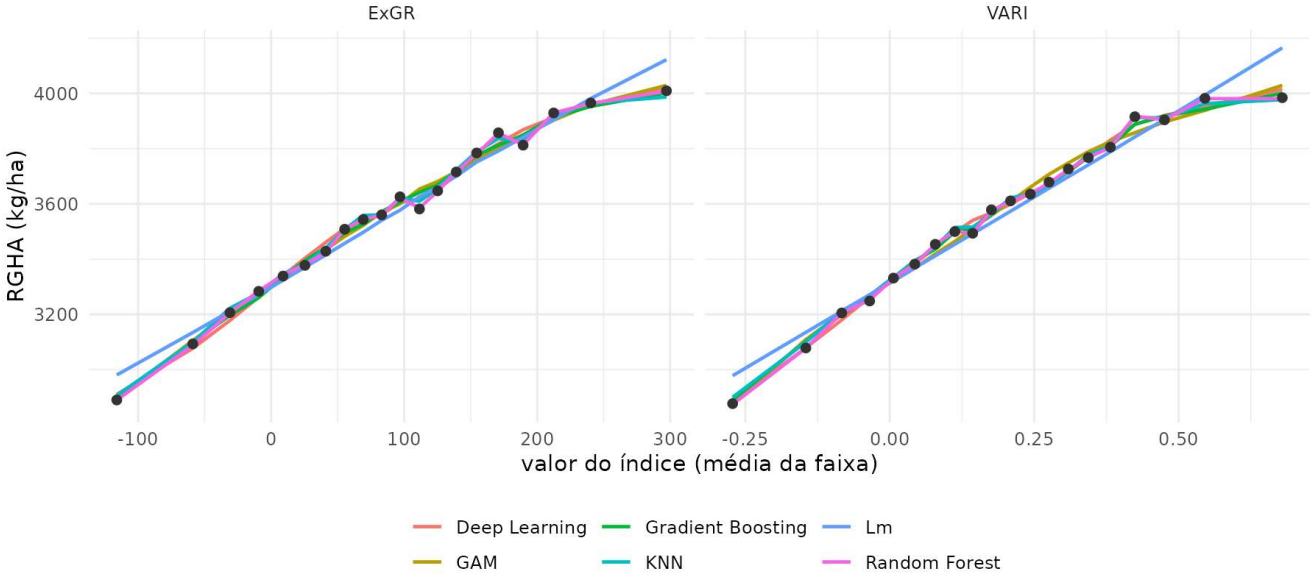


Figura 4. RGHA médio observado (pontos) e predito por método (linhas) em 20 faixas de cada índice, nas fotos de treino. Reta × curva mostra o que cada família consegue representar.

9.2 2. Sobreajuste e estabilidade

Métodos muito flexíveis (floresta de árvores profundas, KNN com k pequeno) tendem a ter R² de treino bem acima do de validação (**gap_R2**) e a variar mais entre partições (**RMSE_dp**). Boosting com profundidade 1, taxa 0,10 e só 100 árvores é fortemente regularizado: não consegue decorar. Compare as colunas: quem tem gap alto aprende o ruído do treino.

```
diag2 <- merge(tab_val[c("Metodo", "R2", "R2_treino", "gap_R2")], tab_rep[c("Metodo", "RMSE", "RMSE_dp")], by = "Metodo", suffixes = c("", "_rep"))
names(diag2) <- c("Metodo", "R2_validacao", "R2_treino", "gap_R2", "RMSE_repeticoes", "RMSE_dp_repeticoes")
diag2 <- merge(diag2, e3[c("Metodo", "razao_RMSE")], by = "Metodo"); names(diag2)[7] <- "razao_RMSE_III"; diag2 <- diag2[order(diag2$RMSE_repeticoes), ]
mostra(diag2, "Sobreajuste (gap_R2 = R² treino − R² validação), estabilidade (desvio do RMSE nas repetições) e transferência para as linhagens (razão_RMSE_III = RMSE no conjunto III / RMSE da validação 20%).", d = 3)
```

Tabela 15. Sobreajuste (gap_R2 = R² treino − R² validação), estabilidade (desvio do RMSE nas repetições) e transferência para as linhagens (razão_RMSE_III = RMSE no conjunto III / RMSE da validação 20%).

Metodo	R2_validacao	R2_treino	gap_R2	RMSE_repeticoes	RMSE_dp_repeticoes	razao_RMSE_III
RNA	0.150	0.175	0.024	684.623	3.440	0.889
GAM	0.150	0.169	0.018	686.410	2.944	0.898
SVM	0.147	0.170	0.022	686.483	3.475	0.895
Deep Learning	0.149	0.171	0.022	686.729	3.312	0.896

Metodo	R2_validacao	R2_treino	gap_R2	RMSE_repeticoes	RMSE_dp_repeticoes	razao_RMSE_III
KNN	0.152	0.173	0.021	688.016	3.239	0.901
nLnn*	0.147	0.164	0.017	688.511	3.259	0.900
Gradient Boosting	0.148	0.165	0.017	690.192	3.224	0.905
Lasso	0.143	0.155	0.011	691.758	3.643	0.910
Bayesian Ridge	0.143	0.155	0.012	691.780	3.635	0.910
BRM	0.143	0.155	0.012	691.781	3.642	0.910
Lm	0.143	0.155	0.012	691.781	3.642	0.910
Ridge	0.145	0.154	0.009	691.989	3.572	0.911
Random Forest	0.032	0.782	0.750	729.730	5.018	0.895

9.3 3. As fotos da mesma parcela são independentes?

Não: as 30 fotos de uma parcela têm o mesmo RGHA. Com partição por fotos, o método pode **reconhecer a parcela** pelos índices e decorar o RGHA dela. Compare as mesmas métricas com partes sorteadas por foto ou por parcela inteira (5 partes). Quanto mais o RMSE sobe ao trocar para a partição por parcela, mais o método dependia dessa memorização.

```
set.seed(CFG$semente); f_lin <- caret::createFolds(I$RGHA, k = 5, list = FALSE)
pm <- tapply(I$RGHA, I$parcela, mean); f_par <- setNames(caret::createFolds(pm, k = 5, list = FALSE), names
(pm))[I$parcela]
cvl <- resume(cv_metodos(I, f_lin)); cvp <- resume(cv_metodos(I, f_par))
ind <- merge(cvl[c("Metodo", "R2", "RMSE")], cvp[c("Metodo", "R2", "RMSE")], by = "Metodo", suffixes = c("_f
otos", "_parcela"))
ind$aumento_RMSE_perc <- 100 * (ind$RMSE_parcela / ind$RMSE_fotos - 1); ind <- ind[order(ind$RMSE_parcela),
]
mostra(ind, "Validação cruzada de 5 partes no conjunto I, sorteando fotos (_fotos) ou parcelas inteiras (_pa
rcela). aumento_RMSE_perc = piora ao deixar parcelas inteiras de fora.", d = 3); salva(ind, "independencia")
```

Tabela 16. Validação cruzada de 5 partes no conjunto I, sorteando fotos (_fotos) ou parcelas inteiras (_parcela). aumento_RMSE_perc = piora ao deixar parcelas inteiras de fora.

Metodo	R2_fotos	RMSE_fotos	R2_parcela	RMSE_parcela	aumento_RMSE_perc
RNA	0.167	685.671	0.144	692.557	1.004
nLnn*	0.160	688.554	0.144	692.825	0.620
GAM	0.164	686.855	0.144	692.883	0.878
Deep Learning	0.164	686.842	0.142	693.684	0.996
SVM	0.163	687.225	0.139	694.682	1.085
KNN	0.160	688.553	0.139	694.920	0.925
Lasso	0.152	691.768	0.138	695.171	0.492
Ridge	0.151	692.028	0.138	695.293	0.472
Bayesian Ridge	0.152	691.808	0.137	695.356	0.513
BRM	0.152	691.806	0.137	695.359	0.514
Lm	0.152	691.806	0.137	695.359	0.514
Gradient Boosting	0.155	690.535	0.133	697.332	0.984
Random Forest	0.059	728.810	0.026	739.016	1.400

9.4 4. Leitura automática desta execução

```
fm <- function(x, d = 1) format(round(x, d), decimal.mark = ",", nsmall = d)
lm_ <- tab_val[tab_val$Metodo == "Lm", ]; gam_ <- tab_val[tab_val$Metodo == "GAM", ]; gb_ <- tab_val[tab_val
$Metodo == "Gradient Boosting", ]
mx <- diag2[which.max(diag2$gap_R2), ]; est <- diag2[which.min(diag2$RMSE_dp_repeticoes), ]; pior <- ind[whi
ch.max(ind$aumento_RMSE_perc), ]
texto("- **Desempenho:** menor RMSE médio em ", CFG$n_rep, " repetições: ", top$Metodo, " (", fm(top$RMSE),
" ± ", fm(top$RMSE_dp), " kg/ha). ", sum(tab_rep$empata_com_o_melhor), " de ", nrow(tab_rep),
      " métodos empatam com ele (até 1 desvio padrão). Método final (", CFG$metodo_final, "): RMSE ", fm(sel
$RMSE), " kg/ha, R² ", fm(sel$R2, 2), ", MAPE ", fm(sel$MAPE), "%.\n",
      "- **Curvatura:** na validação, R² de Lm = ", fm(lm_$R2, 3), ", GAM = ", fm(gam_$R2, 3), " e Gradient
Boosting = ", fm(gb_$R2, 3), ". Diferença GAM − Lm de ", fm(gam_$R2 - lm_$R2, 3),
      ifelse(gam_$R2 - lm_$R2 > 0.01, ": há curvatura que a reta não capta.", ": pouca curvatura captável; a
reta quase empata com as curvas."), "\n",
      "- **Sobreajuste:** maior gap_R2 em ", mx$Metodo, " (", fm(mx$gap_R2, 3), "); Gradient Boosting: ", fm
(gb_$gap_R2, 3), ".\n",
      "- **Estabilidade:** menor desvio do RMSE entre repetições em ", est$Metodo, " (", fm(est$RMSE_dp_repe
ticoes), " kg/ha).\n",
      "- **Independência:** maior piora do RMSE ao deixar parcelas inteiras de fora em ", pior$Metodo, "
(+", fm(pior$aumento_RMSE_perc), "%); Gradient Boosting: +",
      fm(ind$aumento_RMSE_perc[ind$Metodo == "Gradient Boosting"]), "%.")
```

- **Desempenho:** menor RMSE médio em 10 repetições: RNA (684,6 ± 3,4 kg/ha). 5 de 13 métodos empatam com ele (até 1 desvio padrão). Método final (Gradient Boosting): RMSE 690,2 kg/ha, R² 0,16, MAPE 17,4%.
- **Curvatura:** na validação, R² de Lm = 0,143, GAM = 0,150 e Gradient Boosting = 0,148. Diferença GAM − Lm de 0,008: pouca curvatura captável; a reta quase empata com as curvas.
- **Sobreajuste:** maior gap_R2 em Random Forest (0,750); Gradient Boosting: 0,017.
- **Estabilidade:** menor desvio do RMSE entre repetições em GAM (2,9 kg/ha).
- **Independência:** maior piora do RMSE ao deixar parcelas inteiras de fora em Random Forest (+1,4%); Gradient Boosting: +1,0%.

10 Resumo para o artigo

```
fin <- CFG$metodo_final
resumo <- data.frame(etapa = c("repetições 80/20 (média)", "validação cruzada (média)", "validação 20% (1 pa
rtição)", "conjunto II", "conjunto III"),
  R2 = c(sel$R2, tab_cv$R2[tab_cv$Metodo == fin], tab_val$R2[tab_val$Metodo == fin], e2$R2[e2$Metodo == fi
n], e3$R2[e3$Metodo == fin]),
  RMSE = c(sel$RMSE, tab_cv$RMSE[tab_cv$Metodo == fin], tab_val$RMSE[tab_val$Metodo == fin], e2$RMSE[e2$Meto
do == fin], e3$RMSE[e3$Metodo == fin]),
  MAE = c(sel$MAE, tab_cv$MAE[tab_cv$Metodo == fin], tab_val$MAE[tab_val$Metodo == fin], e2$MAE[e2$Metodo ==
fin], e3$MAE[e3$Metodo == fin]),
  MAPE = c(sel$MAPE, tab_cv$MAPE[tab_cv$Metodo == fin], tab_val$MAPE[tab_val$Metodo == fin], e2$MAPE[e2$Meto
do == fin], e3$MAPE[e3$Metodo == fin]))
mostra(cbind(metodo = fin, resumo), paste0("Resumo do método selecionado (", fin, "; preditoras: ", paste(us
ar, collapse = " e "), "). RMSE e MAE em kg ha⁻¹; MAPE em %."), d = 3); salva(resumo, "resumo_metodo_selecio
nado")
```

Tabela 17. Resumo do método selecionado (Gradient Boosting; preditoras: VARI e ExGR). RMSE e MAE em kg ha⁻¹; MAPE em %.

metodo	etapa	R2	RMSE	MAE	MAPE
Gradient Boosting	repetições 80/20 (média)	0.157	690.192	554.199	17.396
Gradient Boosting	validação cruzada (média)	0.156	690.102	554.620	17.420
Gradient Boosting	validação 20% (1 partição)	0.148	692.601	555.415	17.345
Gradient Boosting	conjunto II	0.055	779.475	639.602	25.006
Gradient Boosting	conjunto III	0.209	626.510	494.268	15.918

11 Limites do exemplo

Os números são simulados e servem para mostrar o método; não reproduzem os do Artigo 4. `nLnn*` é um substituto. O Deep Learning é um perceptron em R puro, não Keras. O R² baixo vem do desenho (muita variação de RGHA não ligada à cor do folíolo); com dados reais, confira a fórmula do ExGR, o critério de outliers e a regra de parada do boosting. As explicações de "por que alguns métodos se adaptam melhor" são hipóteses; só a leitura automática e as tabelas vêm dos dados.